9 · Exp 10 — Queue using an array (enqueue, dequeue)

A queue is a line at a ticket counter: join at the back (rear), leave from the front. Two indexes, front and rear, do all the work.

Two ends → enqueue: rear++ → dequeue: front++ → FIFO → linear-queue flaw → circular queue

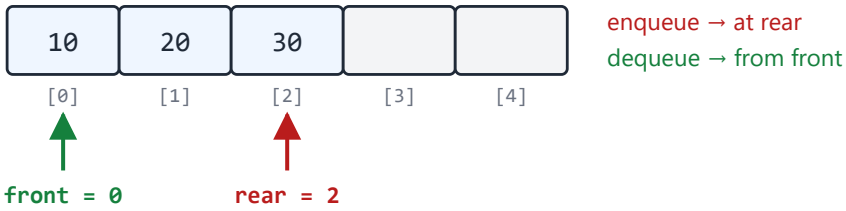
9.1 The idea

A **queue** inserts at the **rear** and removes from the **front**: **FIFO** (First In, First Out). `front` and `rear` both start at **−1** (empty).

GUESS FIRST

MAX = 5. Enqueue 1, 2, 3, 4, 5, dequeue twice, then enqueue 6. There are 2 empty slots. Does the program accept 6?

After enqueue 10, 20, 30 (MAX = 5)



After 1 dequeue:



The manual's example: enqueue 10, 20, 30, then one dequeue.

Operation	Check first	Then
enqueue(v)	`rear == MAX-1` → Overflow	if `front == -1` set `front = 0`; `rear++`; `queueArr[rear] = v`
dequeue()	`front == -1` → Underflow	print `queueArr[front]`; if `front == rear` (last one) reset both to −1, else `front++`
display()	`front == -1` → empty	print `queueArr[front]` … `queueArr[rear]`

WHY RESET TO −1 WHEN THE LAST ELEMENT LEAVES

After dequeuing the only element, front would pass rear, which is confusing. Resetting both to −1 means "empty" is always `front == -1`, and the next enqueue starts again from index 0.

9.2 The program (corrected)

The manual's **listing** (page 5) is right; its **screenshot** (page 9) doesn't compile. In the screenshot, `enqueue(value);` sits after the `if`'s closing brace, so the next `else` has no `if`; choice 3 prints "Program terminated" instead of displaying; and there's a bare `else` with no statement:

G++ SAYS

```
m10_screenshot.cpp:23:9: error: 'else' without a previous 'if'
   23 |         else if (choice == 2)
      |         ^~~~
m10_screenshot.cpp:25:9: error: expected '}' before 'else'
   25 |         else if (choice == 3)
      |         ^~~~
m10_screenshot.cpp:28:5: error: expected primary-expression before '}' token
   28 |     } while (choice != 4);
      |     ^
```

Two more fixes: the array is renamed from `queue` to `queueArr` (with `#include <bits/stdc++.h>`, `queue` clashes with the library's `std::queue`), and the dead `|| front > rear` test is dropped (after the reset rule it can never be true).

E10_QUEUE.CPP

```cpp
#include <iostream>
using namespace std;

#define MAX 5
int queueArr[MAX];         // FIX: renamed from "queue" - clashes with std::queue
int front = -1;
int rear = -1;

void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        cout << "Queue Overflow!" << endl;
    }
    else
    {
        if (front == -1)                    // first element
            front = 0;
        rear++;
        queueArr[rear] = value;
        cout << value << " inserted into queue." << endl;
    }
}

void dequeue()
{
    if (front == -1)
    {
        cout << "Queue Underflow!" << endl;
    }
    else
    {
        cout << queueArr[front] << " deleted from queue." << endl;
        if (front == rear)                  // that was the last element
        {
```

```
            front = -1;
            rear = -1;
        }
        else
        {
            front++;
        }
    }
}

void display()
{
    if (front == -1)
    {
        cout << "Queue is empty." << endl;
    }
    else
    {
        cout << "Queue elements are:" << endl;
        for (int i = front; i <= rear; i++)
            cout << queueArr[i] << " ";
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to enqueue: ";
            cin >> value;
            enqueue(value);
        }
        else if (choice == 2)
        {
            dequeue();
        }
        else if (choice == 3)
        {
            display();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);
```

```
    return 0;
}
```

The manual's Sample Inputs 1 and 2 in one run:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
10 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 20
20 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 30
30 inserted into queue.
Enter your choice: 3
Queue elements are:
10 20 30
Enter your choice: 2
10 deleted from queue.
Enter your choice: 3
Queue elements are:
20 30
Enter your choice: 4
Program terminated.
```

9.3 The linear queue's flaw (a favourite viva question)

Linear queue flaw: enqueue 1..5, dequeue twice, enqueue 7



so the program prints "Queue Overflow!" (verified run). The circular queue fixes this.

Slots freed by dequeue are never reused, because rear only moves forward.

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Queue Underflow!
Enter your choice: 1
Enter value to enqueue: 1
1 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 2
2 inserted into queue.
```

```
Enter your choice: 1
Enter value to enqueue: 3
3 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 4
4 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 5
5 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 6
Queue Overflow!
Enter your choice: 2
1 deleted from queue.
Enter your choice: 2
2 deleted from queue.
Enter your choice: 1
Enter value to enqueue: 7
Queue Overflow!
Enter your choice: 3
Queue elements are:
3 4 5
Enter your choice: 4
Program terminated.
```

Underflow on the empty queue; then 1–5 fill it; 6 overflows (correct, it's full); two dequeues free slots 0 and 1, **yet 7 still overflows**, because rear is still MAX−1. The fix is the **circular queue** (9.5).

WRITE IN YOUR RECORD

Aim: To implement a queue using an array and perform enqueue, dequeue and display operations.

Algorithm: front = rear = −1. *Enqueue:* if rear == MAX−1 → overflow; else (if front == −1, front = 0), rear++, queue[rear] = value. *Dequeue:* if front == −1 → underflow; else print queue[front]; if front == rear, front = rear = −1, else front++. *Display:* print queue[front…rear].

Complexity: enqueue O(1), dequeue O(1), display O(n); space O(MAX).

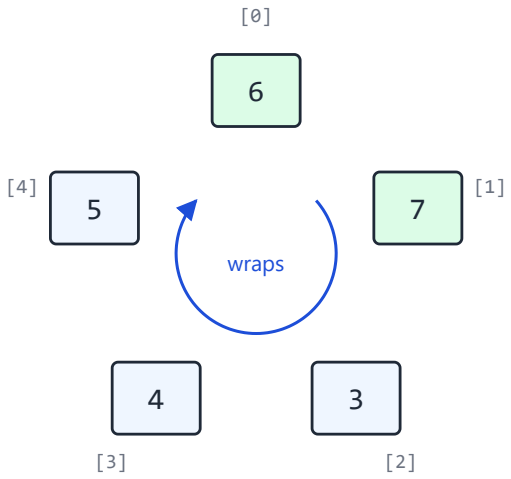
Result: Enqueued 10, 20, 30 (display 10 20 30); dequeue removed 10 (display 20 30), confirming FIFO. Overflow and underflow were detected.

9.4 Viva questions

Question	Answer
What is FIFO?	First In, First Out: elements leave in the order they arrived.
Where is insertion / deletion done?	Insertion at the rear, deletion at the front.
Overflow / underflow conditions?	rear == MAX−1 / front == −1.
Drawback of a linear array queue?	Freed slots at the front are never reused, so it can report overflow while slots are empty.
How does a circular queue fix it?	Indexes wrap: `rear = (rear + 1) % MAX`, so slot 0 is reused after slot MAX−1.
Applications?	CPU and printer scheduling, keyboard buffers, network packets, breadth-first search.
Why are enqueue and dequeue O(1)?	Each touches one index; nothing is shifted (unlike array deletion in Exp 4).

9.5 If the examiner changes the question

Circular queue (full program). The only changes from Exp 10: wrap with `% MAX`, and "full" becomes `(rear + 1) % MAX == front`.




```
front = 2 (value 3)
rear = 1 (value 7)

rear = (rear + 1) % MAX
4 → (4+1) % 5 = 0

full when (rear+1) % MAX == front
```

green = new values stored in the slots the linear queue wasted

After enqueue 1–5, dequeue 1 and 2, enqueue 6 and 7: the new values reuse slots 0 and 1.

E10B_CIRCULAR_QUEUE.CPP

```cpp
// Exp 10 assignment: Circular Queue using an array.
// Same as the linear queue, except rear and front wrap around with  % MAX.
#include <iostream>
using namespace std;

#define MAX 5
int cq[MAX];
int front = -1, rear = -1;
```

```
bool isEmpty() { return front == -1; }
bool isFull()  { return (rear + 1) % MAX == front; }   // next slot after rear is front

void enqueue(int value)
{
    if (isFull())
    {
        cout << "Queue Overflow!" << endl;
        return;
    }
    if (isEmpty())
        front = 0;
    rear = (rear + 1) % MAX;                 // 4 -> 0 : wrap around
    cq[rear] = value;
    cout << value << " inserted into queue." << endl;
}

void dequeue()
{
    if (isEmpty())
    {
        cout << "Queue Underflow!" << endl;
        return;
    }
    cout << cq[front] << " deleted from queue." << endl;
    if (front == rear)                       // last element removed
        front = rear = -1;
    else
        front = (front + 1) % MAX;
}

void display()
{
    if (isEmpty())
    {
        cout << "Queue is empty." << endl;
        return;
    }
    cout << "Queue elements are:" << endl;
    int i = front;
    while (true)
    {
        cout << cq[i] << " ";
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
    cout << endl;
}

int main()
{
    int choice, value;
    do
    {
        cout << "\n1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
```

```
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to enqueue: ";
            cin >> value;
            enqueue(value);
        }
        else if (choice == 2)
            dequeue();
        else if (choice == 3)
            display();
        else if (choice == 4)
            cout << "Program terminated." << endl;
        else
            cout << "Invalid choice!" << endl;
    } while (choice != 4);

    return 0;
}
```

The same sequence that defeated the linear queue. 6 and 7 are now accepted:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 1
1 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 2
2 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 3
3 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 4
4 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 5
5 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 6
Queue Overflow!
Enter your choice: 2
1 deleted from queue.
Enter your choice: 2
2 deleted from queue.
Enter your choice: 1
Enter value to enqueue: 6
6 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 7
7 inserted into queue.
Enter your choice: 3
Queue elements are:
3 4 5 6 7
Enter your choice: 1
```

```
Enter value to enqueue: 8
Queue Overflow!
Enter your choice: 4
Program terminated.
```

Linear vs. circular queue ("WAP to compare"): run both programs on the input above. The linear one reports overflow with free slots; the circular one stores 6 and 7. Both are O(1) per operation.

Peek, isEmpty / isFull, show front and rear (add to Exp 10's program and to the menu):

```
void peek()
{
    if (front == -1)
        cout << "Queue is empty." << endl;
    else
        cout << "Front element is " << queueArr[front] << endl;   // look, don't remove
}
```

```
bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }
```

```
void showFrontRear()
{
    if (front == -1)
        cout << "Queue is empty." << endl;
    else
        cout << "Front = " << queueArr[front] << ", Rear = " << queueArr[rear] << endl;
}
```

Reverse the queue (swap the two ends, move inwards):

```
// Reverse the elements between front and rear: swap the two ends, move inwards
void reverseQueue()
{
    if (front == -1)
        return;
    int i = front, j = rear;
    while (i < j)
    {
        int temp = queueArr[i];
        queueArr[i] = queueArr[j];
        queueArr[j] = temp;
        i++;
        j--;
    }
}
```

Queue using a linked list (read chapter 10 first; no MAX, so no overflow):

```
// Queue using a linked list: no fixed size, so no overflow.
// Enqueue at the rear end of the list, dequeue from the front end.
struct QNode
{
    int data;
    QNode *next;
};
```

```
QNode *qFront = NULL, *qRear = NULL;

void enqueueLL(int value)
{
    QNode *newNode = new QNode;
    newNode->data = value;
    newNode->next = NULL;
    if (qRear == NULL)                      // empty queue
        qFront = qRear = newNode;
    else
    {
        qRear->next = newNode;
        qRear = newNode;
    }
}

int dequeueLL()                             // returns -1 if empty
{
    if (qFront == NULL)
    {
        cout << "Queue Underflow!" << endl;
        return -1;
    }
    QNode *temp = qFront;
    int value = temp->data;
    qFront = qFront->next;
    if (qFront == NULL)                     // queue became empty
        qRear = NULL;
    delete temp;
    return value;
}
```

Queue for CPU scheduling (FCFS). Processes are served in arrival order. Tested: bursts 24, 3, 3 give waiting times 0, 24, 27, average 17.

```
// CPU scheduling (First Come First Served): processes wait in a queue in arrival order.
// Waiting time of a process = sum of burst times of everyone before it.
float fcfs(int burst[], int n)
{
    int waiting = 0, totalWait = 0;
    cout << "Process\tBurst\tWaiting\tTurnaround" << endl;     // \t = tab
    for (int i = 0; i < n; i++)             // dequeue in FIFO order
    {
        int turnaround = waiting + burst[i];
        cout << "P" << i + 1 << "\t" << burst[i] << "\t" << waiting << "\t" << turnaround << endl;
        totalWait += waiting;
        waiting += burst[i];
    }
    cout << "Average waiting time = " << (float)totalWait / n << endl;
    return (float)totalWait / n;
}
```

CHECK 9.1

MAX = 4, linear queue. enqueue 5, enqueue 6, dequeue, dequeue, enqueue 7. What are front and rear after each step?

CHECK 9.2

Circular queue, MAX = 4, front = 2, rear = 3. Enqueue 9: which index is it stored at? Is the queue full now?

Answers

Check 9.1: enqueue 5 → front 0, rear 0. enqueue 6 → 0, 1. dequeue (5) → 1, 1. dequeue (6): front == rear → reset → −1, −1. enqueue 7 → 0, 0.

Check 9.2: rear = (3 + 1) % 4 = **0**, so 9 goes to index 0. Full? (0 + 1) % 4 = 1 ≠ front (2), so **not full**: index 1 is still free.